

Software Project Management

Smart Library Manager (مكتبتي الذكية)

Final Project Report

Name	ID
Omnia Muhammed	24030099
Menna Mahmoud	24030153
Haneen Ibrahim	24030152

1. Executive Summary

This report presents the final outcome of the Smart Library Manager project, a personal library management web application developed by a team of three students over a 3-4 week period as part of the Software Project Management course. The application allows users to organize, search, and track their personal book collection, integrating with the free Open Library API and persisting all data locally in the browser. All planned features were successfully implemented, tested, and delivered within the project's estimated schedule.

2. Introduction

2.1 Background & Problem Statement

Readers who own physical and digital books often struggle to keep track of what they own, what they are currently reading, what they have finished, and what they still want to read. Existing solutions are either too complex, require account creation and internet connectivity, or do not integrate well with free public book databases. This creates a clear opportunity for a lightweight, offline-friendly personal library tracker.

2.2 Project Objectives

- Design and build a fully functional personal library management web application within 3-4 weeks.
- Provide full CRUD (Create, Read, Update, Delete) management of books with reading-status tracking.
- Integrate the Open Library API for smart search and one-click import of book details and covers.
- Provide an interactive statistics dashboard and a complete, exportable activity log.
- Ensure reliable data persistence with no external server or database dependency.
- Apply standard project management practices (WBS, scheduling, cost estimation, risk analysis) throughout development.

3. Project Overview

Item	Details
Project Name	Smart Library Manager (مكتبتي الذكية)
Course	Software Project Management
Team Size	3 members
Duration	21 working days (~4 calendar weeks)
Technology Stack	React, TypeScript, Tailwind CSS, Open Library API, Browser LocalStorage
Project Type	Client-side single-page web application (no backend server required)

4. System Features

4.1 Book Organization & Cataloguing

Users can track each book's reading status (Reading, Finished, Available, or Wishlist) and perform full CRUD operations — adding new books, editing existing entries, deleting books, and updating their status at any time.

4.2 Dual Search & Smart Import

The application supports fast local search within the user's own library, as well as search across the global Open Library API. Selected results can be imported with a single click, automatically pulling in the book's title, author, and cover image.

4.3 Statistics & Activity Log

An interactive dashboard (Bento Grid layout) displays live statistics such as total book counts and status breakdowns. Every action performed in the app is recorded in an activity log, which can be exported as CSV or JSON, or printed directly.

4.4 Flexible Views & Filtering

Users can switch between a card view and a table view, and filter their library by category or reading status. Keyboard shortcuts (Ctrl+K for search, Ctrl+B for adding a new book) allow quick, efficient navigation.

4.5 Persistent Storage & Educational Section

All data is automatically saved to the browser's LocalStorage, ensuring nothing is lost between sessions with no internet connection required after the initial load. An Arabic-language educational section is also included, explaining how the application was built using React, TypeScript, and Tailwind CSS.

5. System Architecture & Technology Stack

Layer	Technology	Purpose
Frontend Framework	React	Component-based UI rendering and state management
Language	TypeScript	Type-safe development, fewer runtime errors
Styling	Tailwind CSS	Utility-first responsive styling
External Data	Open Library API	Book metadata and cover image retrieval
Data Storage	Browser LocalStorage	Client-side persistent data storage

6. Development Methodology

The project followed a phased, WBS-driven approach consisting of five main phases: Project Management, Requirements & Design, Development, Testing, and Deployment & Documentation. A detailed Work Breakdown Structure, Activity List, Gantt Chart, and Critical Path Method (CPM) analysis were produced during planning to guide execution — see the accompanying Project Management Documentation for full details.

Development itself proceeded incrementally: the core CRUD module was built first, followed in parallel by the Open Library API integration, after which the dashboard, activity log, filtering, and persistence layers were layered on top and validated through continuous testing.

7. Testing & Quality Assurance

- Unit and integration testing was performed on each module (CRUD, search/API integration, dashboard, persistence) as it was completed.

- Cross-browser checks were carried out on major browsers (Chrome, Edge, Firefox) to catch rendering or compatibility issues early.
- User acceptance testing confirmed that all acceptance criteria defined in the Project Scope Statement were met before final delivery.
- Bug fixing was carried out iteratively during the dedicated Testing phase, with no critical issues remaining at delivery.

8. Project Management Summary

The project was estimated and tracked using standard project management techniques. The Critical Path Method identified a total project duration of 21 working days, with the critical path running through Project Initiation → Planning → Design → Core Development → Dashboard → Testing → Documentation → Final Reporting. The estimated project cost (Budget at Completion) was \$594 (540 in labor effort plus a 54 contingency reserve), based on a notional academic costing rate.

Earned Value Management (EVM) tracking showed the project fell slightly behind schedule and slightly over budget at the mid-project checkpoint (mainly due to the Open Library API integration taking longer than planned), but the team recovered fully and delivered all planned scope on time by the final deadline.

A full Risk Register was maintained throughout the project, covering risks such as API downtime, data loss, team availability, and scope creep, each with a defined mitigation strategy. No risk materialized into a critical issue during development.

Full details of all project management artifacts (Feasibility Study, Project Charter, Scope Statement, WBS, Activity List, Cost Estimation, EVM, PERT, Gantt Chart, Network Diagram, CPM, and Risk Analysis) are provided in the accompanying Project Management Documentation package.

9. Challenges & Lessons Learned

- Integrating the Open Library API took longer than initially estimated, mainly due to handling inconsistent metadata across different book entries — this was resolved by adding fallback handling for missing fields.
- Coordinating parallel development across three team members required clearer task boundaries than initially planned; using the Activity List and WBS as a shared reference helped reduce overlap and confusion.
- Relying solely on LocalStorage for persistence proved simple and effective for a single-user application, but reinforced the importance of providing an easy data export/backup option for users.
- Applying formal project management tools (CPM, EVM, Risk Register) — even for a small student project — made it much easier to notice the mid-project schedule slip early and correct course before the deadline.

10. Team Contributions

Name	ID	Main Contribution
Omnia Muhammed	24030099	Project leadership, Book CRUD module, Dashboard & statistics development
Menna Mahmoud	24030153	Open Library API integration, search functionality, testing & QA
Haneen Ibrahim	24030152	UI/UX design, project management documentation, activity log & export feature

11. Conclusion & Future Work

The Smart Library Manager project successfully met all of its defined objectives, delivering a complete, functional personal library management application within the planned 3-4 week schedule. The application fulfills a genuine user need with an intuitive interface, smart search and import capability, and reliable local data persistence.

Potential future enhancements include: optional cloud synchronization for multi-device access, a native mobile application, social/sharing features for recommending books between users, and support for additional external book databases beyond Open Library.

12. Project Links

Live Application: <https://bibliotheca-library-management-fron.vercel.app/>

GitHub Repository: [insert the public GitHub repository link here before submission]

13. References

- Open Library API Documentation — <https://openlibrary.org/developers/api>
- React Documentation — <https://react.dev>
- TypeScript Documentation — <https://www.typescriptlang.org/docs>
- Tailwind CSS Documentation — <https://tailwindcss.com/docs>